1. Práctica 6: Medida del tiempo de programa

1.1. Objetivos

- Entender la importancia de tener un *ticker* en el sistema, que provea de una referencia consistente del tiempo.
- Utilizar el *ticker* tanto para esperas como para medidas de tiempo entre eventos.

1.2. Fundamentos teóricos

Los *timers* son una herramienta extremadamente útil para tomar medidas de tiempo. Sin embargo, ofrecen únicamente una medida *local* del tiempo. Es decir, si se dispara un timer sabemos que fue hace un tiempo dado concreto, pero no tenemos más referencia temporal. Cuando se quiere saber el tiempo que pasa entre eventos, lo habitual es usar un *ticker* dentro del sistema que, desde un origen predifinido, va incrementado sistemáticamente un contador cada cierto tiempo dado. Así, mirando el valor del *ticker*, sabremos cuánto tiempo ha pasado desde cualquier otra referencia temporal, con solo calcular la diferencia entre ambos valores.

Hasta ahora, hemos tenido en ocasiones la necesidad de contar tiempo, y para ello hemos utilizado una función similar a esta:

```
void delay(int loops) {
    for (int i = 0; i < loops; i++) {
        __asm("nop");
    }
}
```

Básicamente introducimos un retardo con un bucle que da un cierto número de ciclos antes de terminar. El problema con esta aproximación es doble: por un lado, no sabemos cuánto tarda exactamente una vuelta del bucle (depende, en cierta medida, del número de instrucciones que genere el compilador). Por otro, aunque supiéramos el número de instrucciones, el tiempo que tarda dependerá de la frecuencia del reloj, aciertos en caché... En resumen, no es preciso.

1.3. Desarrollo de la práctica

En esta práctica vamos a utilizar el periférico `SYSTICK` del sistema (`pm0253 4.4`), que nos permite precisamente contar el número de eventos o *ticks* que han ocurrido desde el reseteo del procesador. El tiempo de un *tick* será configurado por nosotros.

El primer paso es definir el periférico junto a su dirección mapeada en memoria:

```
#define SYSTICK_BASE_ADDR       (0xE000E010UL)

typedef struct {
    volatile uint32_t CTRL;     // 0x00
    volatile uint32_t LOAD;     // 0x04
    volatile uint32_t VAL;      // 0x08
    volatile uint32_t CALIB;    // 0x0C
} SysTick_TypeDef;

#define SYSTICK             ((SysTick_TypeDef *) SYSTICK_BASE_ADDR)
```

A continuación debemos crear dos funciones para desactivar y activar el `SYSTICK`. Para ello da el valor adecuado a cada registro. En `LOAD`, ten en cuenta que tendrás que configurar cada cuánto tiempo quieres que se cuente un *tick*. Lo normal es que los *ticks* sean de 1ms. El reloj base es de 16MHz, así que calcula su valor en base a ello. En `VAL` configurarás el valor inicial del contador interno del *ticker*. Finalmente en `CTRL` deberás activar las interrupciones y el periférico, así como establecer la fuente de reloj al oscilador del procesador.

```
static void inline SysTick_Init(void) {
    // Default HSI clock is 16MHz on reset. 1ms tick = 16,000 cycles.
    // ... completar dar los valores adecuados a cada registro.
    SYSTICK->LOAD =
    SYSTICK->VAL  =
    SYSTICK->CTRL =
}

static void inline SysTick_Disable(void) {
    SYSTICK->LOAD = 0U;
    SYSTICK->VAL  = 0U;
    SYSTICK->CTRL = 0U;
}
```

Ahora que tenemos nuestro *ticker* configurado, falta saber en qué *tick* estamos. Para ello, tenemos que declarar una variable que poder consultar.

```
volatile uint32_t msTicks;

static uint32_t inline GetTick(void) {
    return msTicks;
}
```

¿Y... cómo enlazamos esta variable con el *ticker*? Previamente, hemos activado en `CTRL` de `SYSTICK` las interrupciones. Basta con capturar la interrupción del `SYSTICK` y actualizar nuestra variable de conteo:

```
void SysTick_Handler(void) {
    // ... completar añadir un tick a nuestra variable
}
```

Como sabemos que se dispara cada milisegundo, sabemos también que la cuenta de `msTicks` tendrá el valor, en milisegundos, desde que inicializamos `SYSTICK`.

Y ahora sí, estamos listos para sustituir a nuestra función de espera `delay`. Para ello, primero capturaremos el *tick* actual, y simplemente esperaremos a que se cumpla el tiempo aportado:

```
static void inline  Delay_ms(uint32_t ms) {
    // ... completar guardar tick actual y esperar en un bucle a que pasen
    // tantos ticks como se indican
}
```

Como la variable `msTicks` es un entero de 32 bits, tenemos suficiente margen como para contar unos ≈ 50 días. En ese momento, la cuenta volverá a cero. No es un problema en cualquier caso para diferencias relativas de tiempo, donde, aunque hayamos hecho *overflow*, la resta de dos tiempos siempre estará en ese rango de ≈ 50 días.

Con este contador de tiempo preparado, prepara ahora un código donde seas capaz de detectar si hay *dobles clicks* en un botón, tomando las referencias temporales con `GetTick()` cada vez que se pulse, y comprobando que dos consecutivas estén por debajo de los, por ejemplo,

500ms. Recuerda llamar a `Systick_Init()` desde tu código main. No es necesario que actives interrupciones en `NVIC` pues ya están por defecto activadas.

```
void TIM3_IRQHandler(void) {
    // Check if Update Interrupt flag is set
    if (TIM_CHECK_FLAG(TIM3)) {
        // Clear the timer interrupt flag
        TIM_CLEAR_FLAG(TIM3);

        // ... completar comprobar lógica de doble click
        // mirando el estado del botón y el tiempo transcurrido
        // tras la última pulsación

        EXTI_CLEAR_PENDING(0);
        EXTI_UNMASK_INTERRUPT(0, 1);
    }
}
```

1.4. Para hacer más

Hay un "temporizador" especial llamado `RTC` (*Real Time Clock*) capaz de llevar la cuenta de la hora y fecha actual de forma precisa. Con ello, además, puede generar interrupciones a largo plazo. Se utiliza principalmente para realizar esperas largas donde el procesador está dormido (potencialmente durante horas o días) mientras espera la interrupción. Consulta la documentación (`rm0431 25`) y utilízalo para hacer parpadear el led azul (PA5) cada 5 segundos.